

—

 INSTITUTO DE INFORMÁTICA UFG	Universidade Federal de Goiás Instituto de Informática Disciplina: Algoritmos e Estruturas de Dados 2 Prof. André L. Moura	 UFG
---	---	---

SISTEMA DE NAVEGAÇÃO PRIMITIVO

Documentação do Projeto

Integrantes:

Erick Vaz Magalhães
José Borges da Cruz
Matheus Marques Lima
Vitor Albert Gonçalves Pinheiro Avila

Semestre: 2026/1

1. Introdução

1.1 Objetivo do Documento

Este documento apresenta a especificação, arquitetura, requisitos, tecnologias utilizadas e instruções de uso do Sistema de Navegação Primitivo. O sistema foi desenvolvido como projeto final da disciplina de Algoritmos e Estruturas de Dados II, com o objetivo de aplicar conceitos de grafos, estruturas de dados e algoritmos de caminho mínimo.

1.2 Escopo do Projeto e Objetivos

O Sistema de Navegação Primitivo consiste em uma aplicação gráfica capaz de representar mapas e redes viárias através de grafos ponderados, permitindo ao usuário encontrar o menor caminho entre dois pontos utilizando o algoritmo de Dijkstra.

Os principais objetivos do sistema são:

- Representar ruas e avenidas como grafos.
- Permitir importação de mapas reais.
- Encontrar caminhos mínimos entre dois pontos.
- Exibir visualmente os resultados obtidos.
- Permitir criação e edição de grafos.
- Demonstrar a aplicação prática de estruturas de dados estudadas na disciplina.

1.3 Visão Geral do Sistema

O sistema possui interface gráfica interativa que permite:

- Importação de mapas reais e arquivos de grafos (.poly, .osm e .txt).
- Criação e edição manual de vértices e arestas.
- Seleção de vértices de origem e destino.
- Execução do algoritmo de Dijkstra.
- Visualização do menor caminho encontrado.
- Exibição de estatísticas de execução.
- Cópia da visualização do grafo para a área de transferência.

2. Arquitetura do Sistema

2.1 Visão Arquitetural

O sistema foi desenvolvido seguindo uma arquitetura modular baseada na separação de responsabilidades, dividida em camadas com baixo acoplamento. A camada de domínio (core) não depende da interface nem dos leitores de arquivo, o que permite testar o algoritmo de forma isolada.

A arquitetura é dividida em:

- Camada de Apresentação (UI) — src/ui
- Camada de Domínio (Core) — src/core
- Módulos Utilitários — src/loaders e src/utis

Essa divisão facilita a manutenção, a reutilização de código e a evolução futura do sistema.

Estrutura de diretórios:

```
main.py
src/
├── core/      grafo, heap mínima e Dijkstra (implementações próprias)
├── loaders/   leitura de arquivos .poly, .osm e .txt
├── ui/        janela principal, canvas de desenho e índice espacial
└── utils/     geometria, medição de desempenho e área de transferência
```

2.2 Descrição das Camadas

2.2.1 Camada de Apresentação (UI)

Responsável pela interação entre o usuário e o sistema. Principais responsabilidades:

- Exibir grafos por meio de desenho direto (QPainter).
- Capturar eventos do mouse (clique, arraste e roda).
- Gerenciar barra de ferramentas e opções.
- Apresentar resultados e estatísticas.

2.2.2 Camada de Domínio (Core)

Responsável pelas regras de negócio do sistema. Principais responsabilidades:

- Representação dos grafos por lista de adjacência.
- Gerenciamento de vértices e arestas.
- Execução do algoritmo de Dijkstra com heap mínima.
- Cálculo de rotas e reconstrução do caminho.

2.2.3 Módulos Utilitários

Responsáveis pelas funcionalidades auxiliares. Principais responsabilidades:

- Importação de arquivos (.poly, .osm e .txt).
- Conversão de coordenadas geográficas para cartesianas (UTM).
- Cópia da imagem para a área de transferência.
- Medição de desempenho da execução.
- Cálculos geométricos (distância euclidiana).

2.3 Tecnologias Utilizadas

Linguagem de programação:

- Python 3.12 (compatível também com versões mais recentes)

Bibliotecas:

- PyQt6 (interface gráfica e desenho do grafo)

Observação: o núcleo do sistema (representação do grafo, heap mínima e algoritmo de Dijkstra), bem como a leitura dos formatos .poly, .osm e .txt, foi implementado pela equipe, sem o uso de bibliotecas externas de grafos. A única dependência externa é a PyQt6, utilizada para a interface gráfica.

Ferramentas:

- Visual Studio Code

- Git
- GitHub
- PyInstaller (geração do executável)

3. Detalhamento dos Módulos

3.1 Módulo Core (src/core)

Responsável pela lógica principal do sistema. Componentes:

Graph (graph.py)

Representa o grafo através de lista de adjacência (`list[list[Edge]]`), indexada pelo identificador do vértice. Suporta grafos direcionados e não direcionados e oferece operações de adicionar/remover vértices e arestas. A remoção de vértice utiliza marcação de inativo (tombstone) para preservar os identificadores existentes.

Vertex

Representa um vértice do grafo (identificador e coordenadas x, y). Utiliza `__slots__` para reduzir o consumo de memória.

Edge

Representa uma aresta do grafo (vértice de destino, peso e indicador de mão única). O peso é a distância euclidiana entre os vértices.

MinHeap (min_heap.py)

Fila de prioridade implementada como heap binária mínima sobre vetor, utilizada pelo Dijkstra. Operações de inserção e remoção em $O(\log n)$. Implementação própria.

Dijkstra (dijkstra.py)

Implementa o algoritmo de caminho mínimo utilizando a heap mínima, com lazy deletion (descarte de entradas obsoletas) e parada antecipada ao atingir o destino. Retorna distância total, caminho, número de nós explorados e tempo de processamento.

3.2 Módulo UI (src/ui)

Responsável pela interface gráfica. Componentes:

Main Window (main_window.py)

Janela principal da aplicação. Reúne a barra de ferramentas, o canvas e o painel lateral de estatísticas, além de orquestrar abertura de arquivos, execução do algoritmo e modos de exibição/edição.

Graph Canvas (graph_canvas.py)

Área de desenho do grafo. Faz a renderização com QPainter (com culling de viewport para desempenho), pan e zoom, seleção de origem/destino, destaque da rota e edição do grafo via mouse.

Spatial Grid (spatial_grid.py)

Índice espacial em grade uniforme que localiza o vértice mais próximo de um clique em tempo quase constante, evitando varredura linear em grafos grandes.

3.3 Módulos Utilitários (src/loaders e src/utils)

POLY Loader (loaders/poly_loader.py)

Importação de arquivos .poly (saída do conversor de mapas), contendo vértices e arestas.

OSM Loader (loaders/osm_loader.py)

Importação direta de arquivos .osm do OpenStreetMap, convertendo coordenadas geográficas (latitude/longitude) para cartesianas UTM (zona 23S).

TXT Loader (loaders/txt_loader.py)

Importação de arquivos .txt no formato Origem, Destino, Peso.

Geometry (utils/geometry.py)

Funções geométricas auxiliares (distância euclidiana e bounding box).

Performance (utils/performance.py)

Medição de tempo de execução.

Clipboard (utils/clipboard.py)

Cópia da imagem do grafo para a área de transferência.

4. Funcionalidades Detalhadas

4.1 Requisitos Funcionais

RF01 – Importar mapas reais

Permite importar arquivos .poly, .osm e .txt contendo informações de vértices e arestas e convertê-los para grafos utilizáveis pelo sistema.

RF02 – Enumerar vértices e rotular arestas

Permite exibir os identificadores dos vértices e os pesos (distâncias) associados às arestas.

RF03 – Selecionar origem e destino

Permite escolher visualmente os vértices de origem e destino para cálculo do caminho mínimo, com marcação em cores distintas (origem em verde, destino em azul) e possibilidade de desfazer a seleção.

RF04 – Exibir rota do menor caminho

Calcula e destaca visualmente, em vermelho, o menor caminho encontrado pelo algoritmo de Dijkstra.

RF05 – Criar e editar grafos

Permite adicionar ou remover vértices e arestas por meio da interface gráfica, com auxílio do mouse.

RF06 – Suporte a grafos direcionados e não direcionados

Permite representar vias de mão única (arestas com seta) e mão dupla (linhas), bem como alternar o modo do grafo entre direcionado e não direcionado.

RF07 – Exibir estatísticas de execução

Apresenta o tempo de processamento, o custo total da rota e a quantidade de vértices explorados.

RF08 – Copiar imagem do grafo

Permite copiar a visualização atual do grafo para a área de transferência.

4.2 Requisitos Não Funcionais

ID	Descrição
RNF01	Utilização de cores distintas para vértices e arestas.
RNF02	Representação visual diferenciada para vias de mão única (seta) e mão dupla (linha).
RNF03	Suporte eficiente a grafos de grande porte, por meio de lista de adjacência, heap mínima e culling de renderização.
RNF04	Tempo de resposta inferior a dois segundos para grafos médios (cerca de 500 nós). Nos testes realizados, o cálculo para 500 nós ficou abaixo de 1 ms.
RNF05	Uso eficiente de memória, por meio de lista de adjacência e do uso de __slots__ nas estruturas.
RNF06	Interface intuitiva e amigável.
RNF07	Compatibilidade com Windows e Linux.
RNF08	Código modular, documentado e de fácil manutenção.

5. Estruturas de Dados Utilizadas

5.1 Lista de Adjacência

Estrutura principal utilizada para armazenar o grafo (`list[list[Edge]]`). Vantagens:

- Menor consumo de memória.
- Melhor desempenho em grafos esparsos.

5.2 Heap Mínima

Utilizada como fila de prioridade do algoritmo de Dijkstra (implementação própria em vetor). Complexidade: $O(\log n)$ para inserção e remoção.

5.3 Vetor de Distâncias

Armazena a menor distância conhecida entre a origem e cada vértice.

5.4 Vetor de Predecessores

Permite reconstruir o caminho mínimo encontrado.

5.5 Conjunto de Visitados

Impede o reprocessamento de vértices já explorados.

6. Formatos de Dados de Entrada

6.1 Arquivo TXT

Formato: Origem, Destino, Peso. Exemplo:

```
A,B,10
A,C,15
B,D,8
```

6.2 Arquivo OSM

Arquivos exportados do OpenStreetMap contendo nós (latitude/longitude) e vias (sequências de nós). O sistema converte as coordenadas geográficas em cartesianas (UTM, zona 23S) e monta o grafo automaticamente.

6.3 Arquivo POLY

Arquivo gerado pelo conversor de mapas, contendo os vértices (identificador e coordenadas cartesianas x, y) e as arestas (pares de vértices). O peso de cada aresta é calculado como a distância euclidiana entre os vértices. Estrutura:

```
nVertices 2 0 1
id x y          (* nVertices)
...
nArestas 1
id origem destino 0 (* nArestas)
...
0
```

7. Guia de Instalação e Execução

7.1 Pré-Requisitos

- Windows 10 ou superior (ou Linux)
- Python 3.12 ou superior
- Conexão com internet para instalação das dependências

7.2 Instalação

Executar:

```
pip install -r requirements.txt
```

7.3 Execução

Executar:

```
python main.py
```

7.4 Geração do Executável

Executar:

```
pyinstaller --onefile --windowed --name SistemaNavegacao main.py
```

O executável é gerado na pasta dist/.

8. Guia de Uso da Aplicação

8.1 Interação com o Canvas

O usuário pode dar zoom com a roda do mouse, mover o mapa arrastando com o botão esquerdo e selecionar vértices com o clique. No modo de edição, é possível criar vértices (clique em área vazia), criar arestas (clique em dois vértices) e remover vértices (botão direito).

8.2 Encontrar Caminho Mínimo

Passos:

1. Abrir um arquivo de mapa/grafos.
2. Selecionar o vértice de origem (verde).
3. Selecionar o vértice de destino (azul).
4. Executar o algoritmo de Dijkstra ("Traçar menor caminho").
5. Visualizar o caminho destacado em vermelho e as estatísticas no painel lateral.

8.3 Importação, Edição e Exportação

A aplicação permite:

- Importar grafos (.poly, .osm e .txt).
- Editar grafos existentes.
- Alternar entre grafo direcionado e não direcionado.
- Copiar a visualização para a área de transferência.

9. Conclusão

O Sistema de Navegação Primitivo demonstra a aplicação prática dos conceitos de grafos e algoritmos de caminho mínimo estudados na disciplina de Algoritmos e Estruturas de Dados II. A utilização do algoritmo de Dijkstra, associada a estruturas eficientes como listas de adjacência e heaps mínimas implementadas pela própria equipe, permite obter desempenho adequado mesmo em grafos de grande porte, atendendo aos requisitos funcionais e não funcionais especificados.